Pear / Holepunch P2P Resolver Threat Model

This document outlines the security assumptions, privacy rules, and threat vectors for the SatsPath PearResolver integration.

1. Overview

The PearResolver allows SatsPath to resolve aliases (e.g., `alice@example.com`) to `SignedPaymentProfile`s over the Holepunch/Hyperswarm peer-to-peer network. This bypasses centralized servers and DNS, relying on distributed hash tables (DHT) to locate peers announcing a specific topic.

2. Privacy Rules (P2P-03)

To prevent passive scraping of the Hyperswarm DHT, the following privacy rules are strictly enforced:

1. **No Plaintext Aliases in Topics**: The Hyperswarm topic used for discovery MUST NOT be the plaintext alias. It must be a cryptographic hash of the alias (e.g., `SHA256(alias)`).
2. **No Private Keys in Pear Node**: The `satspath-pear` daemon or script only handles public `SignedPaymentProfile` JSON objects. It never loads, generates, or transmits the user's `identity_secret_key` or any wallet secrets.
3. **Fail-Closed Resolution**: If a peer returns malformed data or a profile belonging to a different alias, it is dropped silently.

3. Threat Vectors and Mitigations

Threat	Mitigation	Layer
Passive DHT Scraping	Topics are `SHA256(alias)`, making it impossible to harvest a list of registered emails/aliases by simply listening to the DHT.	Pear / JS
Profile Spoofing / Man-in-the-Middle	The Rust resolver verifies the `secp256k1` signature of every profile returned by a peer against the declared `identity_pubkey`. A malicious peer cannot forge a signature for an identity they do not control.	Rust Core
Replay Attacks	The `SignedPaymentProfile` includes an `expires_at` timestamp. The Rust resolver rejects expired profiles.	Rust Core
Denial of Service (DHT Spam)	An attacker could spam the topic with junk data. The `satspath-pear` resolver will disconnect from peers sending non-JSON or invalid schema data. The Rust resolver handles validation quickly and drops invalid signatures.	Pear + Rust
Alias Collision (Intentional)	Two users cannot easily claim the same alias because the sender verifies the profile signature against the expected identity, or trust is established via out-of-band exchange (TOFU or verified channels).	Rust Core

4. Trust Model

- **Hyperswarm Network**: Untrusted. Any node can join and announce any topic.
- **satspath-pear Process**: Trusted (runs locally on the user's machine).
- **Profile Data**: Untrusted until the Rust Core validates the `secp256k1` signature and expiration.